Groth16 검증 단계 (Verification Phase) 상세 분석

목차

1. 검증 단계 개요
2. 검증 키 전처리
3. 공개 입력 준비
4. 페어링 기반 검증
5. 페어링 수학적 원리
6. 배치 검증 최적화
7. 검증 성능 분석
8. 보안 분석

검증 단계 개요

검증 단계는 Groth16의 가장 효율적인 부분으로, 회로 크기와 무관하게 일정한 시간에 증명의 유효성을 확인할 수 있습니다. 이는 3개의 페어링 연산만으로 복잡한 계산의 정확성을 검증할 수 있기 때문입니다.

검증 과정 개요

```
pub fn verify_proof(
pvk: &PreparedVerifyingKey<E>,
```

```
proof: &Proof<E>,

public_inputs: &[E::ScalarField],

) -> R1CSResult<bool> {

        // 1. 공개 입력 준비

        let  prepared_inputs = Self::prepare_inputs(pvk, public_inputs)?;

        // 2. 전처리된 입력으로 검증

        Self::verify_proof_with_prepared_inputs(pvk, proof, &prepared_inputs)

}
```

검증의 핵심 아이디어

페어링 방정식:

```
e(A, B) = e(α, β) · e(L_ic, γ) · e(C, δ)
```

이 방정식이 성립하면 증명이 유효하며, 다음을 보장합니다:

- **정확성**: 올바른 계산이 수행되었음
- **완전성**: 모든 제약 조건이 만족되었음
- **건전성**: 거짓 명제에 대해서는 증명 생성 불가

검증 시간 복잡도

| 연산 | 시간 복잡도 | 설명 |

|------|----------|------|

| 공개 입력 처리 | $O(\ell)$ | ℓ = 공개 입력 개수 |

| 페어링 계산 | O(1) | 상수 시간 (3개 페어링) |

| 전체 | O(ℓ) | 공개 입력에만 비례 |

검증 키 전처리

전처리의 목적

검증 키 전처리는 반복적인 검증에서 성능을 향상시키기 위해 일부 계산을 미리 수행하는 과정입니다.

```
pub fn prepare_verifying_key<E: Pairing>(vk: &VerifyingKey<E>) -> PreparedVeri
    PreparedVerifyingKey {
        vk: vk.clone(),
        // e(α*G, β*H) 미리 계산
        alpha_g1_beta_g2: E::pairing(vk.alpha_g1, vk.beta_g2).0,
        // -γ*H를 페어링에 최적화된 형태로 전처리
        gamma_g2_neg_pc: vk.gamma_g2.into_group().neg().into_affine().in
        // -δ*H를 페어링에 최적화된 형태로 전처리
        delta_g2_neg_pc: vk.delta_g2.into_group().neg().into_affine().in
    }
}
```

각 전처리 요소의 역할

1. alpha_g1_beta_g2 미리 계산:

```
alpha_g1_beta_g2: E::pairing(vk.alpha_g1, vk.beta_g2).0
```

효과:

- 검증 시마다 `e(α, β)` 계산 생략
- 약 25% 검증 시간 단축
- 메모리 사용량 약간 증가 (GT 원소 1개)

2. G2 원소 전처리:

```
gamma_g2_neg_pc: vk.gamma_g2.into_group().neg().into_affine().into()

delta_g2_neg_pc: vk.delta_g2.into_group().neg().into_affine().into()
```

전처리 과정:

1. **부호 반전**: `gamma_g2 → -gamma_g2`
2. **좌표 변환**: Projective → Affine (필요시)
3. **페어링 최적화**: Miller 루프에 최적화된 형태로 변환

음수 사용 이유:

원래 방정식을 변형하여 계산 효율성 향상:

```
원래: e(A, B) = e(α, β) · e(L_ic, γ) · e(C, δ)

변형: e(A, B) · e(L_ic, -γ) · e(C, -δ) = e(α, β)
```

전처리 비용 분석

```
// 전처리 시간 측정

let prep_time = start_timer!(|| "Prepare verifying key");

let pvk = prepare_verifying_key(&vk);
```

```
end_timer!(prep_time);


// 일반적인 비용 (BLS12-381 기준)
struct PreprocessingCost {
        pairing_computation: Duration, // ~2ms (1회)
        g2_preparation: Duration, // ~0.1ms (2회)
        total: Duration, // ~2.2ms
}
```

비용 대비 효과:

- **일회성 비용**: 전처리는 한 번만 수행
- **반복 이익**: 검증할 때마다 25% 시간 절약
- **손익분기점**: 약 4-5회 검증 후 이익

공개 입력 준비

공개 입력 처리 과정

```
pub fn prepare_inputs(
pvk: &PreparedVerifyingKey<E>,
public_inputs: &[E::ScalarField],
) -> R1CSResult<E::G1> {
        // 1. 입력 개수 검증
        if (public_inputs.len() + 1) != pvk.vk.gamma_abc_g1.len() {
```

```
            return Err(SynthesisError::MalformedVerifyingKey);
        }

        // 2. 선형 결합 계산
        // L_ic = γ^(-1) * (β*a_0 + α*b_0 + c_0) * G +
        // Σ(public_input[i] * γ^(-1) * (β*a_i + α*b_i + c_i) * G)
        let mut g_ic = pvk.vk.gamma_abc_g1[0].into_group();
        for (input, gamma_abc_i) in public_inputs.iter()
        .zip(pvk.vk.gamma_abc_g1.iter().skip(1)) {
            g_ic.add_assign(&gamma_abc_i.mul_bigint(input.into_bigint()));
        }

        Ok(g_ic)
}
```

수학적 의미 분석

gamma_abc_g1의 구조:

```
// 설정 단계에서 계산됨
gamma_abc_g1[i] = γ^(-1) * (β * a_i(τ) + α * b_i(τ) + c_i(τ)) * G
```

선형 결합 계산:

```
L_ic = Σ(public_input[i] * gamma_abc_g1[i])
```

```
= Σ(public_input[i] * γ^(-1) * (β*a_i + α*b_i + c_i) * G)

= γ^(-1) * Σ(public_input[i] * (β*a_i + α*b_i + c_i)) * G
```

입력 검증 및 에러 처리

```
// 상세한 입력 검증
pub fn validate_public_inputs(
vk: &VerifyingKey<E>,
public_inputs: &[E::ScalarField],
) -> R1CSResult<()> {
        // 개수 확인
        let expected_len = vk.gamma_abc_g1.len() - 1;
        if public_inputs.len() != expected_len {
                return Err(SynthesisError::MalformedVerifyingKey);
        }
                // 범위 확인 (필요시)
        for (i, input) in public_inputs.iter().enumerate() {
                if input >= &E::ScalarField::MODULUS {
                        eprintln!("Public input {} is out of range", i);
                        return Err(SynthesisError::InvalidInput);
                }
        }
        Ok(())
```

```
}
```

최적화된 MSM 사용

```
// 대량의 공개 입력이 있을 때 MSM 사용
pub fn prepare_inputs_msm(
pvk: &PreparedVerifyingKey<E>,
public_inputs: &[E::ScalarField],
) -> R1CSResult<E::G1> {
        if public_inputs.is_empty() {
                return Ok(pvk.vk.gamma_abc_g1[0].into_group());
        }
        // MSM으로 한 번에 계산
        let scalars: Vec<_> = public_inputs.iter()
        .map(|s| s.into_bigint())
        .collect();
        let bases = &pvk.vk.gamma_abc_g1[1..];
        let msm_result = E::G1::msm_bigint(bases, &scalars);
        Ok(pvk.vk.gamma_abc_g1[0].into_group() + msm_result)
}
```

MSM 사용 기준:

- **적은 입력 (< 10개)**: 순차 계산이 더 효율적
- **많은 입력 (≥ 10개)**: MSM이 더 효율적

- **임계값**: 구현과 하드웨어에 따라 조정

페어링 기반 검증

핵심 검증 함수

```
pub fn verify_proof_with_prepared_inputs(
pvk: &PreparedVerifyingKey<E>,
proof: &Proof<E>,
prepared_inputs: &E::G1,
) -> R1CSResult<bool> {
        // 다중 Miller 루프 계산
        // e(A, B) * e(L_ic, -γ*H) * e(C, -δ*H) 계산
        let qap = E::multi_miller_loop(
                [
                        <E::G1Affine as Into<E::G1Prepared>>::into(proof.a),
                        prepared_inputs.into_affine().into(),
                        proof.c.into(),
                ],
                [
                        proof.b.into(),
                        pvk.gamma_g2_neg_pc.clone(),
                        pvk.delta_g2_neg_pc.clone(),
                ],
```

```
        );

        // 최종 지수화
        let  test = E::final_exponentiation(qap)
        .ok_or(SynthesisError::UnexpectedIdentity)?;

        // e(A, B) * e(L_ic, -γ*H) * e(C, -δ*H) = e(α*G, β*H) 확인
        Ok(test.0 == pvk.alpha_g1_beta_g2)
}
```

페어링 계산 과정 분석

1. Miller 루프 단계:

```
let  qap = E::multi_miller_loop(
[proof.a.into(), prepared_inputs.into_affine().into(), proof.c.into()],
[proof.b.into(), pvk.gamma_g2_neg_pc.clone(), pvk.delta_g2_neg_pc.clone()],
);
```

계산 내용:

- `miller_loop(proof.a, proof.b)`
- `miller_loop(prepared_inputs, -gamma_g2)`
- `miller_loop(proof.c, -delta_g2)`
- 세 결과의 곱

2. 최종 지수화:

```
let  test = E::final_exponentiation(qap)?;
```

목적: Miller 루프 결과를 GT의 소수 부분군으로 매핑

페어링 방정식의 수학적 검증

원래 방정식:

```
e(A, B) = e(α, β) · e(L_ic, γ) · e(C, δ)
```

변형된 방정식 (음수 사용):

```
e(A, B) · e(L_ic, -γ) · e(C, -δ) = e(α, β)
```

페어링 성질 활용:

```
e(P, -Q) = e(P, Q)^(-1) = e(-P, Q)
```

따라서:

```
e(A, B) · e(L_ic, γ)^(-1) · e(C, δ)^(-1) = e(α, β)

e(A, B) / (e(L_ic, γ) · e(C, δ)) = e(α, β)

e(A, B) = e(α, β) · e(L_ic, γ) · e(C, δ) ✓
```

에러 처리 및 예외 상황

```
// 상세한 에러 처리
pub fn verify_with_detailed_errors(
pvk: &PreparedVerifyingKey<E>,
proof: &Proof<E>,
public_inputs: &[E::ScalarField],
) -> R1CSResult<bool> {
        // 1. 증명 유효성 검사
        if proof.a.is_zero() || proof.b.is_zero() || proof.c.is_zero() {
                eprintln!("Proof contains zero elements");
                return Err(SynthesisError::MalformedProof);
        }
        // 2. 공개 입력 준비
        let prepared_inputs = Self::prepare_inputs(pvk, public_inputs)
        .map_err(|e| {
        eprintln!("Failed to prepare inputs: {:?}", e);
        e
        })?;
        // 3. 페어링 계산
        let qap = E::multi_miller_loop(/* ... */);
        let test = E::final_exponentiation(qap)
        .ok_or_else(|| {
                eprintln!("Final exponentiation failed - possible invalid proof"
```

```
                SynthesisError::UnexpectedIdentity
        })?;
        // 4. 결과 비교
        let  is_valid = test.0 == pvk.alpha_g1_beta_g2;
        if !is_valid {
                eprintln!("Pairing equation does not hold");
                eprintln!("Expected: {:?}", pvk.alpha_g1_beta_g2);
                eprintln!("Got: {:?}", test.0);
        }
        Ok(is_valid)
}
```

페어링 수학적 원리

페어링의 정의와 성질

페어링 함수:

```
e: G1 × G2 → GT
```

핵심 성질:

1. **쌍선형성**: `e(aP, bQ) = e(P, Q)^(ab)`
2. **비퇴화성**: `e(P, Q) ≠ 1` (P, Q ≠ 0일 때)
3. **효율적 계산**: 다항식 시간에 계산 가능

Groth16에서 페어링 활용

지식 추출 원리:

```
// 증명자가 다음을 만족하는 A, B, C를 생성했다면:
// e(A, B) = e(α, β) · e(L_ic, γ) · e(C, δ)

// 페어링의 성질에 의해 다음이 성립:
// A = α + Σ(a_i * u_i) + r * δ (어떤 r에 대해)
// B = β + Σ(b_i * u_i) + s * δ (어떤 s에 대해)
// C = Σ(c_i * u_i) + H(τ)*Z(τ)/δ + A*s + B*r - r*s*δ
```

건전성 보장:

- 올바른 witness 없이는 위 구조를 만족하는 A, B, C 생성 불가
- 페어링 방정식이 성립하려면 QAP 관계식도 만족해야 함

타원곡선별 페어링 구현

BLS12-381 곡선:

```
impl Pairing for Bls12_381 {
        type G1 = G1Projective;
        type G2 = G2Projective;
        type GT = Fq12;
        fn  pairing(p: G1Affine, q: G2Affine) -> GT {
                // Optimal Ate 페어링 구현
```

```
                let miller_result = miller_loop(&[(p, q)]);

                final_exponentiation(miller_result)

        }

}
```

성능 특성:

- **Miller 루프**: 약 80% 시간 소모
- **최종 지수화**: 약 20% 시간 소모
- **전체 시간**: 약 1-2ms (단일 페어링)

배치 검증 최적화

배치 검증의 필요성

여러 증명을 동시에 검증할 때 개별 검증보다 효율적인 방법이 필요합니다.

```
pub fn batch_verify(

pvk: &PreparedVerifyingKey<E>,

proofs: &[Proof<E>],

public_inputs: &[Vec<E::ScalarField>],

) -> R1CSResult<bool> {

        if proofs.len() != public_inputs.len() {

                return Err(SynthesisError::InvalidInput);

        }

        // 무작위 선형 결합을 통한 배치 검증
```

```
        let mut rng = thread_rng();

        let random_coeffs: Vec<E::ScalarField> = (0..proofs.len())

        .map(|_| E::ScalarField::rand(&mut rng))

        .collect();

        // 각 증명을 무작위 계수로 결합

        let combined_proof = combine_proofs(proofs, &random_coeffs);

        let combined_inputs = combine_public_inputs(public_inputs, &random_coef

        // 결합된 증명 검증

        Self::verify_proof(pvk, &combined_proof, &combined_inputs)

}
```

무작위 선형 결합 기법

기본 아이디어:

```
각 증명 i에 대해: e(A_i, B_i) = e(α, β) · e(L_i, γ) · e(C_i, δ)


무작위 계수 r_i로 결합:

Π e(A_i, B_i)^r_i = Π (e(α, β) · e(L_i, γ) · e(C_i, δ))^r_i


페어링의 쌍선형성에 의해:

e(Σ r_i·A_i, Σ r_i·B_i) = e(α, β)^(Σ r_i) · e(Σ r_i·L_i, γ) · e(Σ r_i·C_i, δ)
```

배치 검증 구현

```
fn combine_proofs<E: Pairing>(
proofs: &[Proof<E>],
coeffs: &[E::ScalarField],
) -> Proof<E> {
let mut combined_a = E::G1::zero();
let mut combined_b = E::G2::zero();
let mut combined_c = E::G1::zero();
for (proof, coeff) in proofs.iter().zip(coeffs.iter()) {
        combined_a += proof.a * coeff;
        combined_b += proof.b * coeff;
        combined_c += proof.c * coeff;
}
Proof {
        a: combined_a.into_affine(),
        b: combined_b.into_affine(),
        c: combined_c.into_affine(),
        }
}


fn combine_public_inputs<F: PrimeField>(
inputs: &[Vec<F>],
coeffs: &[F],
```

```
) -> Vec<F> {
        if inputs.is_empty() {
                return Vec::new();
        }
        let input_len = inputs[0].len();
        let mut combined = vec![F::zero(); input_len];
        for (input_vec, coeff) in inputs.iter().zip(coeffs.iter()) {
                for (combined_elem, input_elem) in combined.iter_mut().zip(inpu
                        *combined_elem += *input_elem * coeff;
                }
        }
        combined
}
```

배치 검증 성능 분석

시간 복잡도:

- **개별 검증**: O(n) (n = 증명 개수)
- **배치 검증**: O(n) + O(1) ≈ O(n) (하지만 상수가 작음)

실제 성능 향상:

```
// 벤치마크 결과 (100개 증명 기준)
struct BatchVerificationBenchmark {
        individual_verification: Duration, // ~200ms
```

```
        batch_verification: Duration, // ~50ms
        speedup: f64, // ~4x
}
```

메모리 사용량:

- **추가 메모리**: O(n) (무작위 계수 저장)
- **임시 메모리**: O(1) (결합된 증명 저장)

검증 성능 분석

단일 검증 성능

```
// 성능 측정 코드
pub fn benchmark_verification<E: Pairing>(
pvk: &PreparedVerifyingKey<E>,
proof: &Proof<E>,
public_inputs: &[E::ScalarField],
iterations: usize,
) -> Duration {
        let start = Instant::now();
        for _ in 0..iterations {
                let result = Self::verify_proof(pvk, proof, public_inputs).unwr
                assert!(result);
        }
```

```
        start.elapsed() / iterations  as u32
}
```

성능 분해 분석:

```
struct VerificationBreakdown {
        input_preparation: Duration, // ~10% (공개 입력 처리)
        miller_loops: Duration, // ~70% (3개 Miller 루프)
        final_exponentiation: Duration, // ~20% (최종 지수화)
        total: Duration, // ~2ms (BLS12-381 기준)
}
```

곡선별 성능 비교

| 곡선 | 보안 수준 | 검증 시간 | 증명 크기 |

|------|----------|----------|----------|

| BN254 | ~100비트 | ~1.5ms | 128바이트 |

| BLS12-381 | ~128비트 | ~2.0ms | 192바이트 |

| BLS12-377 | ~128비트 | ~2.2ms | 192바이트 |

하드웨어별 성능

```
// 다양한 환경에서의 성능 측정
struct HardwarePerformance {
        desktop_cpu: Duration, // Intel i7: ~1.8ms
```

```
        server_cpu: Duration, // Xeon: ~1.5ms
        mobile_cpu: Duration, // ARM: ~5ms
        embedded: Duration, // Raspberry Pi: ~20ms
}
```

최적화 기법:

1. **SIMD 활용**: 벡터 연산으로 필드 연산 가속화
2. **어셈블리 최적화**: 중요한 루프를 어셈블리로 구현
3. **캐시 최적화**: 메모리 접근 패턴 최적화
4. **병렬 처리**: 독립적인 연산들을 병렬로 처리

보안 분석

건전성 (Soundness) 분석

건전성 정리:

거짓 명제에 대해서는 유효한 증명을 생성할 확률이 무시할 수 있을 정도로 작습니다.

```
// 건전성 위반 시나리오
enum SoundnessViolation {
        // 잘못된 witness로 유효한 증명 생성
        FalseWitness,
        // QAP 관계식을 만족하지 않는 증명
        InvalidQAP,
        // 페어링 방정식을 우회하는 공격
```

```
        PairingAttack,
}
```

보안 가정:

1. **q-SDH 가정**: q-Strong Diffie-Hellman 문제의 어려움
2. **q-PKE 가정**: q-Power Knowledge of Exponent 가정
3. **DLOG 가정**: 이산로그 문제의 어려움

완전성 (Completeness) 분석

완전성 정리:

올바른 witness를 가진 정직한 증명자는 항상 유효한 증명을 생성할 수 있습니다.

```
// 완전성 테스트
#[test]
fn test_completeness() {
        let mut rng = test_rng();
        let (pk, vk) = Groth16::setup(TestCircuit::default(), &mut rng).unwrap(
        let pvk = prepare_verifying_key(&vk);
        // 올바른 witness로 증명 생성
        for _ in 0..100 {
                let circuit = TestCircuit::new_random(&mut rng);
                let proof = Groth16::prove(&pk, circuit.clone(), &mut rng).unw
                // 검증은 항상 성공해야 함
                assert!(Groth16::verify_with_processed_vk(
```

```
                        &pvk,
                        &circuit.public_inputs(),
                        &proof
                ).unwrap());
        }
}
```

영지식성 (Zero-Knowledge) 분석

영지식성 정리:

증명으로부터 witness에 대한 정보를 추출할 수 없습니다.

```
// 시뮬레이터 존재성 테스트
fn simulate_proof<E: Pairing>(
vk: &VerifyingKey<E>,
public_inputs: &[E::ScalarField],
rng: &mut impl Rng,
) -> Proof<E> {
        // 실제 witness 없이도 동일한 분포의 증명 생성
        // (단, 페어링 검증은 실패)
        let r = E::ScalarField::rand(rng);
        let s = E::ScalarField::rand(rng);
        // 무작위 증명 생성 (시뮬레이션)
        Proof {
```

```
                a: (vk.alpha_g1 + E::G1::rand(rng)).into_affine(),
                b: (vk.beta_g2 + E::G2::rand(rng)).into_affine(),
                c: E::G1::rand(rng).into_affine(),
        }
}
```

부채널 공격 방지

```
// 상수 시간 구현
fn constant_time_verify<E: Pairing>(
pvk: &PreparedVerifyingKey<E>,
proof: &Proof<E>,
public_inputs: &[E::ScalarField],
) -> R1CSResult<bool> {
        // 분기 없는 구현으로 타이밍 공격 방지
        let prepared_inputs = Self::prepare_inputs(pvk, public_inputs)?;
        // 상수 시간 페어링 계산
        let qap = E::multi_miller_loop(/* ... */);
        let test = E::final_exponentiation(qap)?;
        // 상수 시간 비교
        Ok(constant_time_eq(&test.0, &pvk.alpha_g1_beta_g2))
}
```

```rust
fn constant_time_eq<T: PartialEq>(a: &T, b: &T) -> bool {
    // 타이밍 공격을 방지하는 상수 시간 비교
    use subtle::ConstantTimeEq;
    // 실제 구현에서는 더 복잡한 로직 필요
    a == b
}
```

보안 고려사항:

1. **타이밍 공격**: 검증 시간이 입력에 의존하지 않도록 구현
2. **전력 분석**: 전력 소모 패턴으로 정보 유출 방지
3. **캐시 공격**: 메모리 접근 패턴 일정하게 유지
4. **결함 주입**: 하드웨어 오류를 통한 공격 방지

검증 단계는 Groth16의 가장 효율적이고 실용적인 부분으로, 복잡한 계산의 정확성을 간단한 페어링 연산으로 확인할 수 있게 해줍니다. 이는 블록체인과 같은 분산 환경에서 특히 중요한 특성이며, Groth16이 널리 채택되는 주요 이유 중 하나입니다.